Datos-a-escala

HOW TO:
DRAW A HORSE

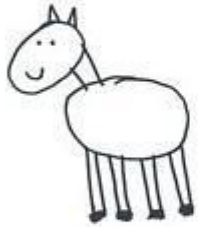
BY VAN OKTOP



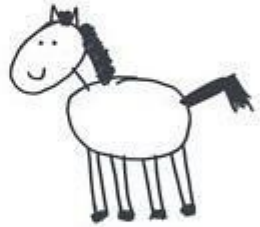
① DRAW 2 CIRCLES



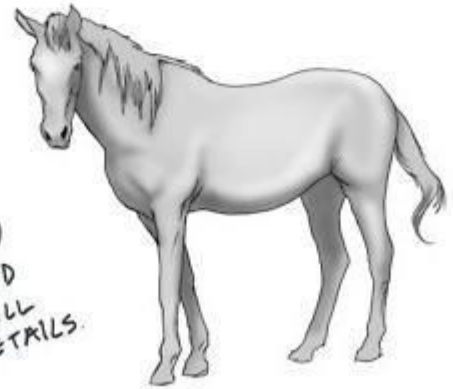
② DRAW THE LEGS



③ DRAW THE FACE



④ DRAW THE HAIR



⑤ ADD SMALL DETAILS.

HOW TO:
DRAW A HORSE

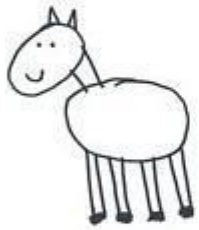
BY VAN OKTOP



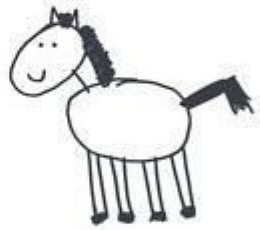
① DRAW 2 CIRCLES



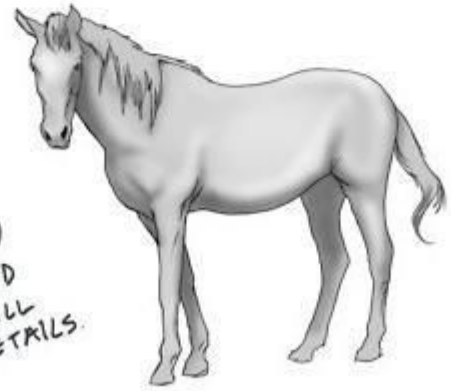
② DRAW THE LEGS



③ DRAW THE FACE



④ DRAW THE HAIR



⑤ ADD SMALL DETAILS.

Cómo se llega a “tener muchos datos”

~~MVP~~ vibecoding

Creces un poquito y te das cuenta que la base de datos no es magia del todo.

Asumamos una base de datos de verdad, Postgres

Índices, un poco de orden, alguna CPU más, memoria para caché que no falte y a funcionar otro par de años

Company	Article and reported scale
Instagram	"What Powers Instagram: Hundreds of Instances, Dozens of Technologies" (Dec. 2011). At **14M+ users**, Instagram said its main Postgres shard cluster used **12 database instances plus 12 replicas**; photos were stored separately on S3, which held **several terabytes**. A related post, "Sharding & IDs at Instagram", says they were handling **over 25 photos and 90 likes per second** as they began sharding. medium.com +2
Figma	"The growing pains of database architecture" says that in **2020** Figma used one large RDS database for most metadata. Database traffic grew about **3× annually**, and peak CPU was above **65%**. The follow-up "How Figma's databases team lived to tell the scale" reports the stack had grown nearly **100× since 2020**; some tables reached **several TB and billions of rows**. By the end of 2022, Figma had a dozen vertically partitioned databases, plus caching and read replicas. Figma Blog +1
ChatGPT / OpenAI	"Scaling PostgreSQL to power 800 million ChatGPT users" reports a **single primary PostgreSQL instance** with nearly **50 read replicas** across regions—not one database instance in total. OpenAI says it supported **800M users** and **millions of queries per second**; PostgreSQL load had grown more than **10×** over the prior year. The post doesn't give the database's storage size. OpenAI

Podemos decir que no necesitamos pensar demasiado hasta tener algún terabyte

Sí, pero

No todos los terabytes son iguales

buscar una aguja en un pajar

vs

contar todas las pajas del pajar

buscar una aguja en un pajar

==

última vez que fulano se conectó

contar todas las pajas del pajar

==

media de tiempo de uso el año pasado

Sales and Marketing Sample

Share Dashboard

Ask a question about the data on this dashboard

Get me started

Total Volume
IN 2014

50K

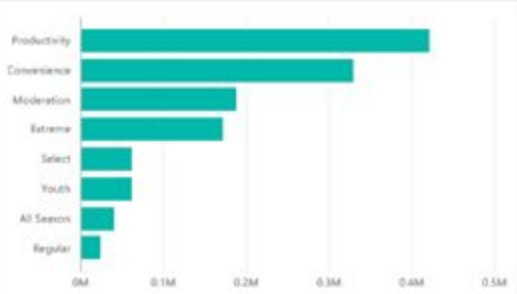
Market Share
LAST 12 MONTHS

32.86%

% Units Market Share vs. % Units Market Share Rolling 12 Months
BY MONTH



Total Units Overall
BY SEGMENT



Our Total Volume
IN 2014

16K

Sentiment

68

Total Units YTD Variance %
BY MONTH, MANUFACTURER



Total Units YTD
BY MANUFACTURER, REGION



Sentiment Gap

4

Total Units for 2014
BY MONTH, MANUFACTURER



Total Units
1M

Industry % Market Share YOY Change
BY ROLLING PERIOD, REGION



Enganchas el BI a tu base de datos de producción y...

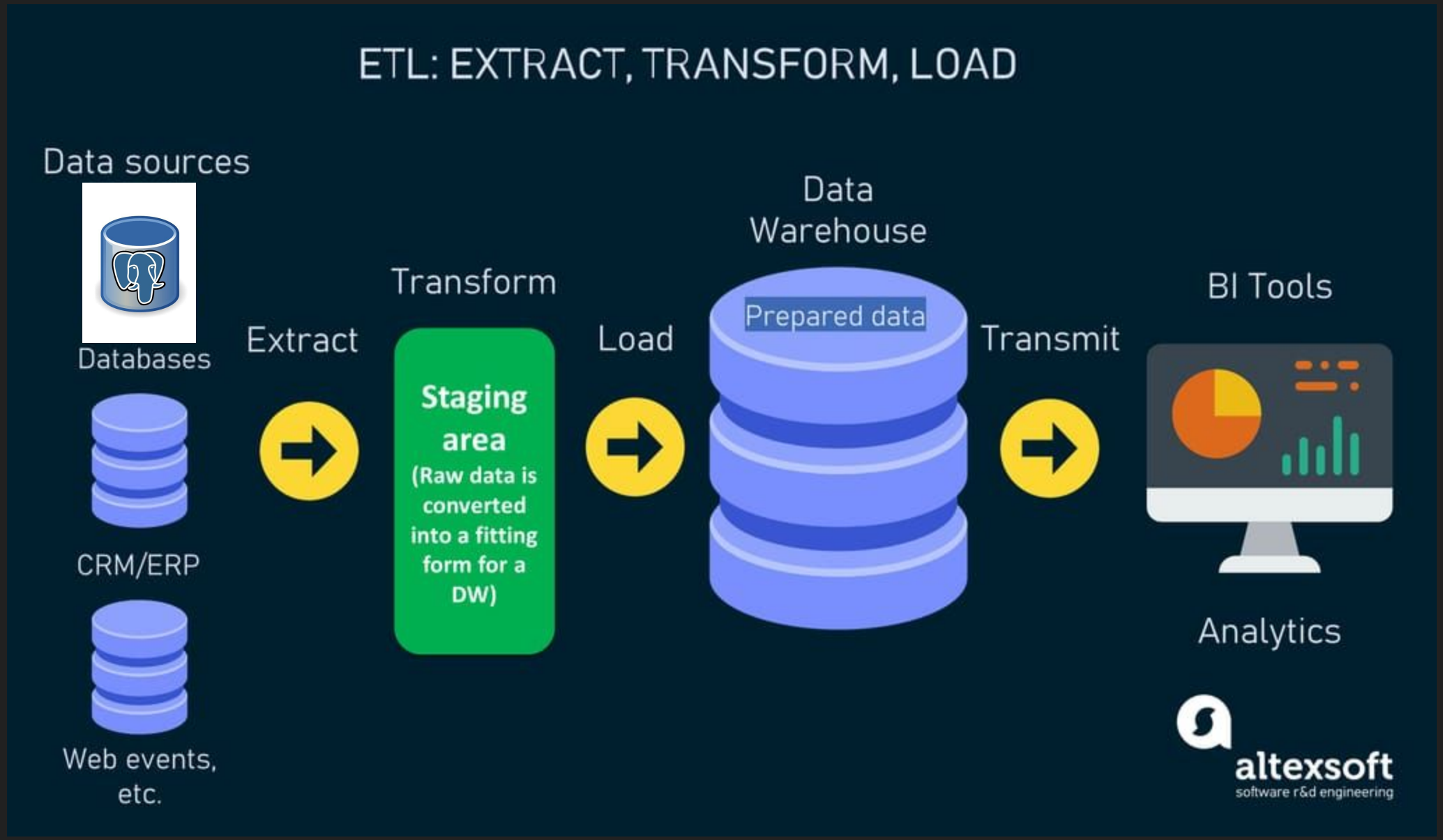
Muere… añades réplicas de lectura, optimizas queries, buscas índices nuevos pensando que van a solucionar algo, creas vistas, procesos batch, limitas el uso del BI, te quejas al CEO, el CEO dice que es tu problema, le dices que va a costar dinero, él no entiende, le pregunta a chatGPT y le da una solución que no sirve pero te jode vivo

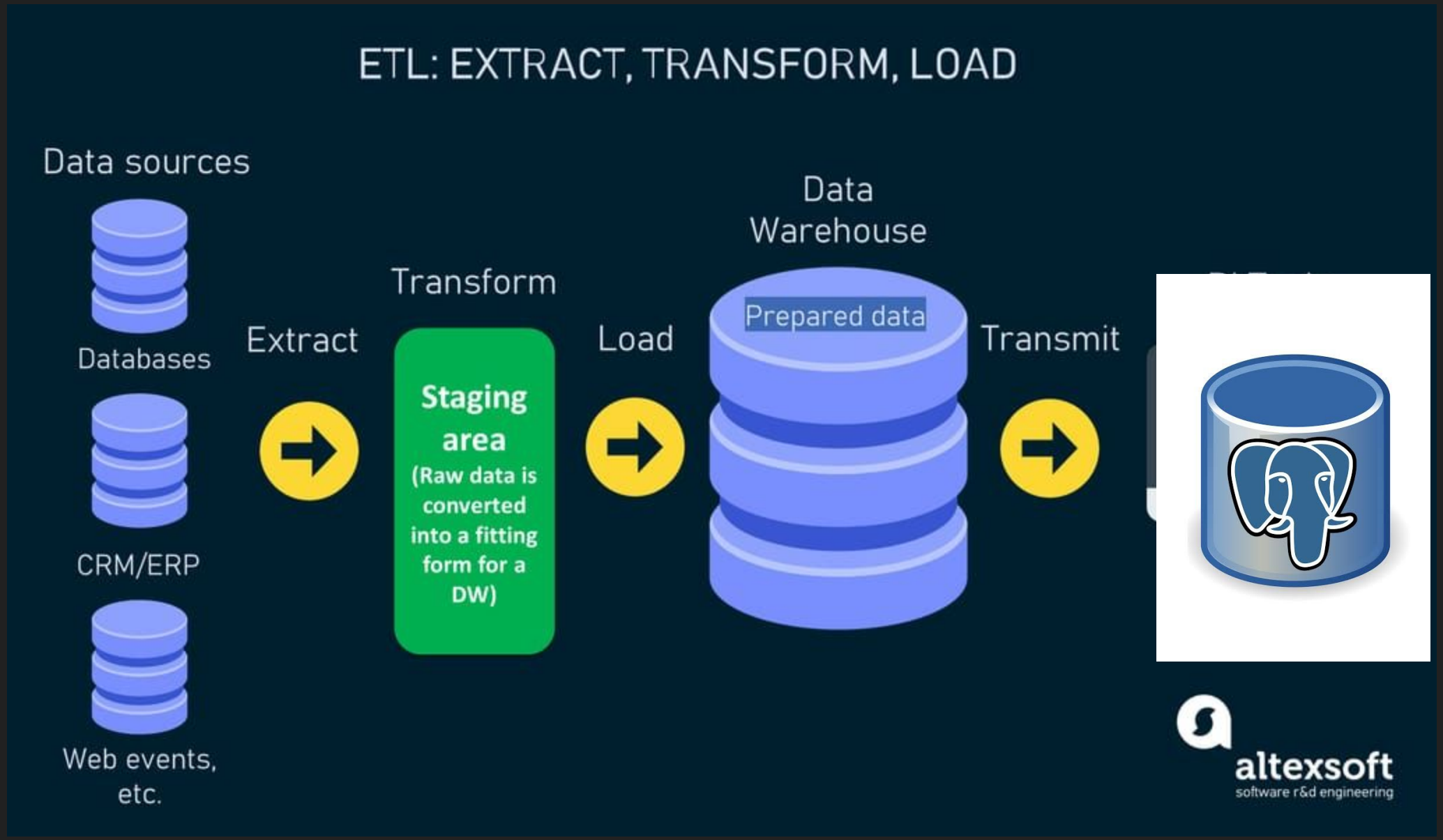
Entonces alguien te dice lo que sospechabas: necesitas una herramienta diferente

EL DATAWAREHOUSE

Ok, entonces empiezas a dividir los datos, mandas cada tipo de dato a un sistema diferente

transaccionales -> Postgres
analíticos -> DWH (Snowflake, BigQuery, ClickHouse)


ETL: EXTRACT, TRANSFORM, LOAD
Data sources
Databases
CRM/ERP
Web events, etc.
Extract
Transform
Staging area
(Raw data is converted into a fitting form for a DW)
Load
Data Warehouse
Prepared data
Transmit
BI Tools
Analytics
altexsoft
software r&d engineering


ETL: EXTRACT, TRANSFORM, LOAD
Data sources
Databases
CRM/ERP
Web events, etc.
Extract
Transform
Staging area
(Raw data is converted into a fitting form for a DW)
Load
Data Warehouse
Prepared data
Transmit
altexsoft
software r&d engineering

Y qué se hace con esos datos que están en el “warehouse”

- reporting: cuanta gente me visitó ayer?
- llevar la cuenta de cosas: cuanto tengo que cobrarte?
- logging: qué pasó hace 5 días
- “intelligence”: cuánto debería cobrarte para maximizar mi beneficio?
- feedback-driven system: cambio el comportamiento en función de lo que pasa

Pero nadie te explica el porqué y entonces te das cuenta que hay dos tipos de personas

MINI CURSO DE 120 SEGUNGOS

FUNDAMENTOS DE SISTEMAS DE DATOS


MONDAY
TUESDAY
WEDNESDAY
THURSDAY



	session_id	timestamp	source_ip
Row 1	1331246660	3/8/2012 2:44PM	99.155.155.225
Row 2	1331246351	3/8/2012 2:38PM	65.87.165.114
Row 3	1331244570	3/8/2012 2:09PM	71.10.106.181
Row 4	1331261196	3/8/2012 6:46PM	76.102.156.138

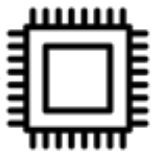
Traditional Memory Buffer

Row 1	1331246660
	3/8/2012 2:44PM
	99.155.155.225
Row 2	1331246351
	3/8/2012 2:38PM
	65.87.165.114
Row 3	1331244570
	3/8/2012 2:09PM
	71.10.106.181
Row 4	1331261196
	3/8/2012 6:46PM
	76.102.156.138

Arrow Memory Buffer

session_id	1331246660
	1331246351
	1331244570
	1331261196
timestamp	3/8/2012 2:44PM
	3/8/2012 2:38PM
	3/8/2012 2:09PM
	3/8/2012 6:46PM
source_ip	99.155.155.225
	65.87.165.114
	71.10.106.181
	76.102.156.138

```
SELECT * FROM clickstream
WHERE session_id = 1331246351
```



Intel CPU

> Write less. Store less. Read less.

> Write, keep, and read related data together.

> Sort. Compress. Preprocess. Mark.

> O(1) preferred. O(n) acceptable. O(n^2) forbidden.

CIERRE DEL MINI CURSO

ENHORABUENA, ESTÁIS EN EL PERCENTIL 95 EN CONOCIMIENTO DE TRABAJO CON DATOS

HAGO ENTREGA DE LOS DIPLOMAS AL TERMINAR

	session_id	timestamp	source_ip
Row 1	1331246660	3/8/2012 2:44PM	99.155.155.225
Row 2	1331246351	3/8/2012 2:38PM	65.87.165.114
Row 3	1331244570	3/8/2012 2:09PM	71.10.106.181
Row 4	1331261196	3/8/2012 6:46PM	76.102.156.138

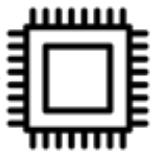
Traditional Memory Buffer

Row 1	1331246660
	3/8/2012 2:44PM
	99.155.155.225
Row 2	1331246351
	3/8/2012 2:38PM
	65.87.165.114
Row 3	1331244570
	3/8/2012 2:09PM
	71.10.106.181
Row 4	1331261196
	3/8/2012 6:46PM
	76.102.156.138

Arrow Memory Buffer

session_id	1331246660
	1331246351
	1331244570
	1331261196
timestamp	3/8/2012 2:44PM
	3/8/2012 2:38PM
	3/8/2012 2:09PM
	3/8/2012 6:46PM
source_ip	99.155.155.225
	65.87.165.114
	71.10.106.181
	76.102.156.138

```
SELECT * FROM clickstream
WHERE session_id = 1331246351
```



Intel CPU

Modern hardware: sequential scans are 1000x faster

> write less data
> store less data.
> read less data.
> keep related data together.
> sort the data.
> compress it.
> prefer O(1), accept O(n), avoid $O(n^2)$.



42
42
44
46
40



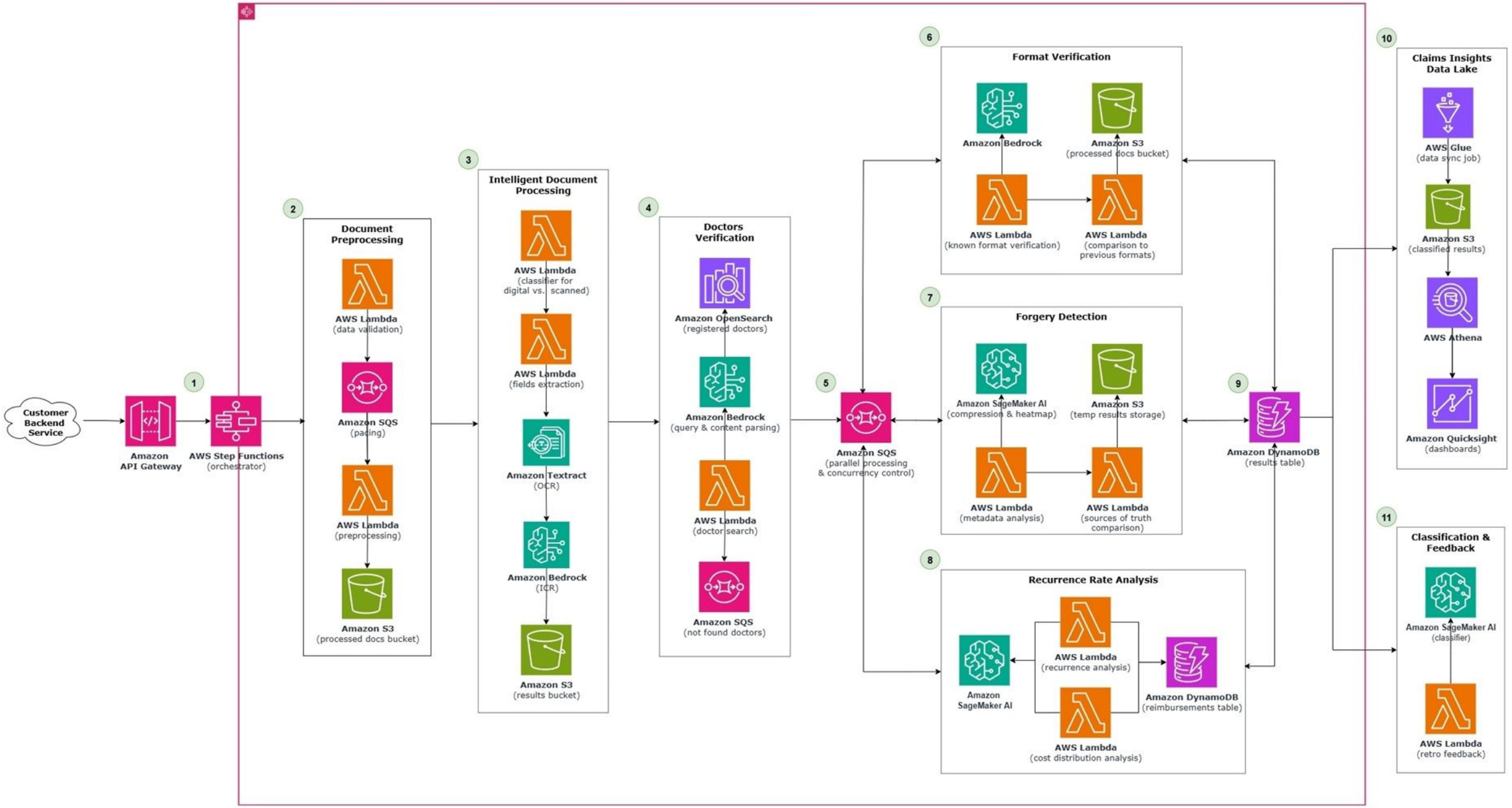
CIERRE DEL MINI CURSO

ENHORABUENA, ESTÁIS EN EL PERCENTIL 95 EN CONOCIMIENTO DE TRABAJO CON DATOS

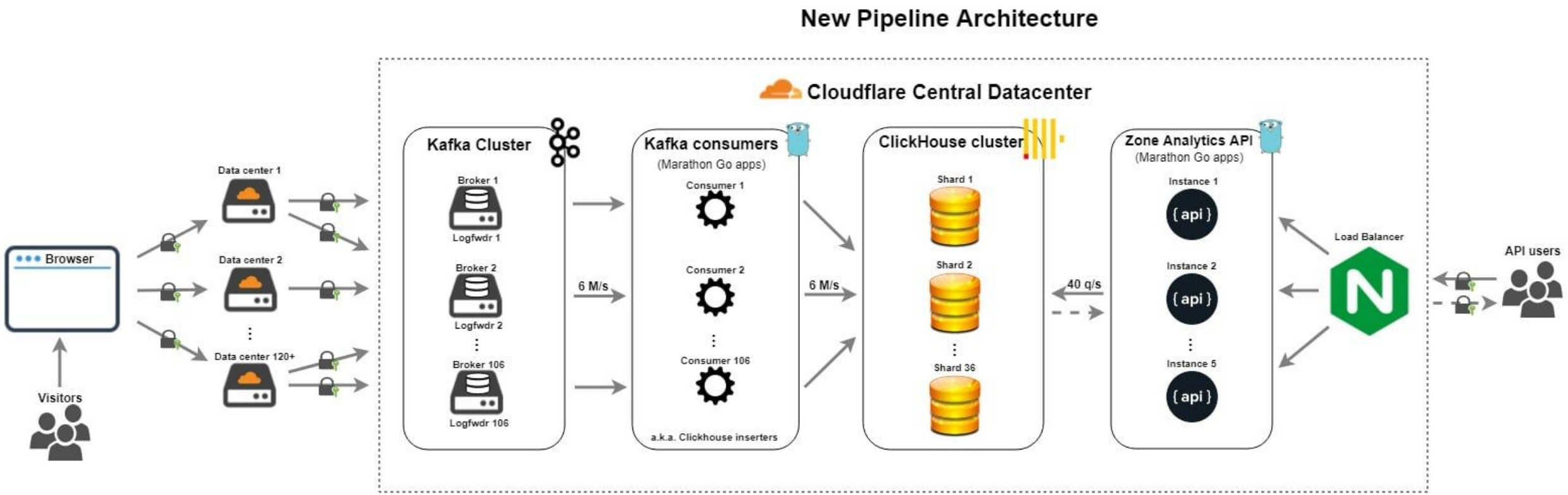
HAGO ENTREGA DE LOS DIPLOMAS AL TERMINAR

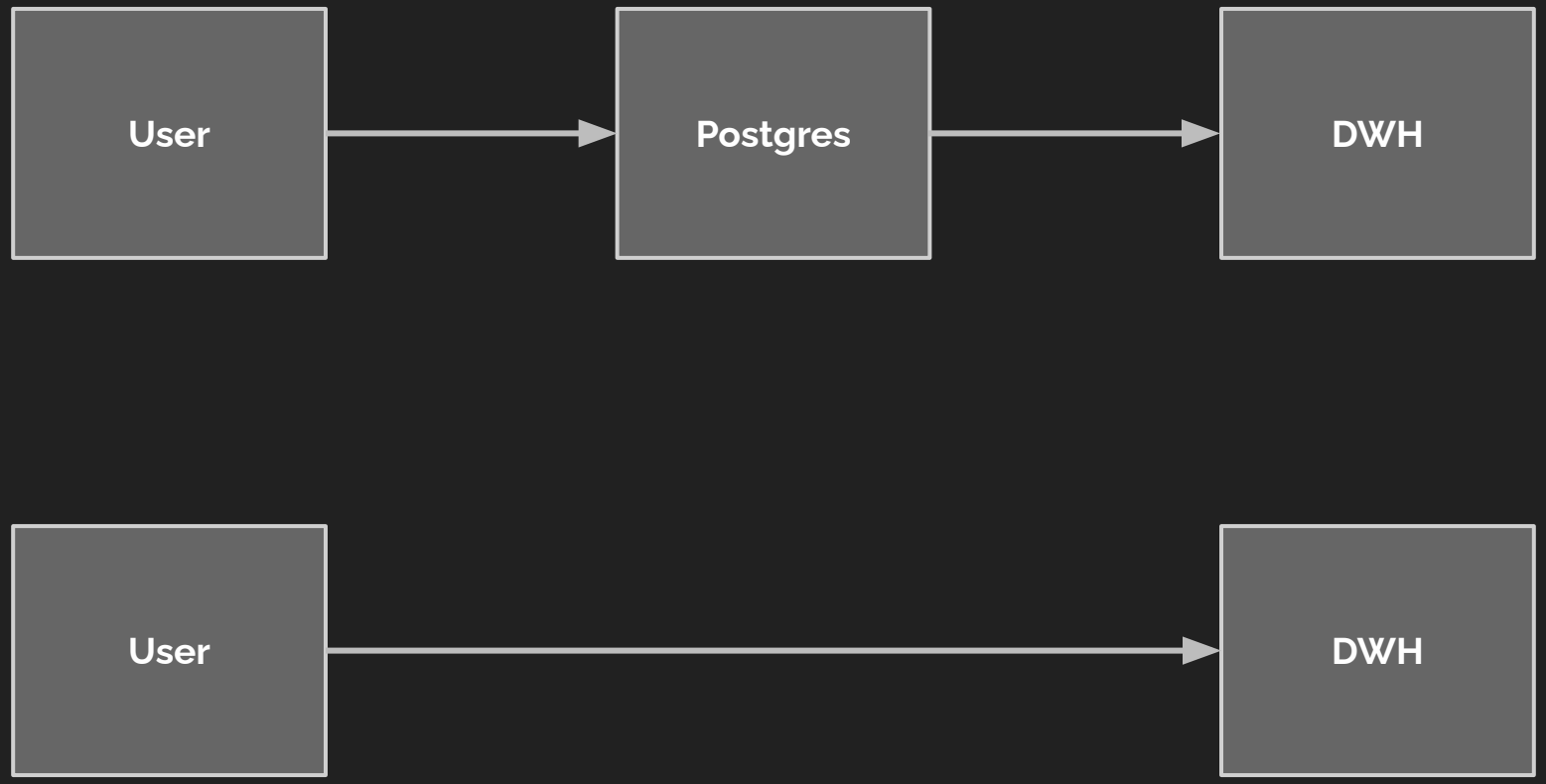
Creces y creces y tu arquitectura ya no sirve. Entonces es cuando planteas tu arquitectura de datos “desde cero”

Y alguien te dice lo que sospechabas: necesitas algo que escale

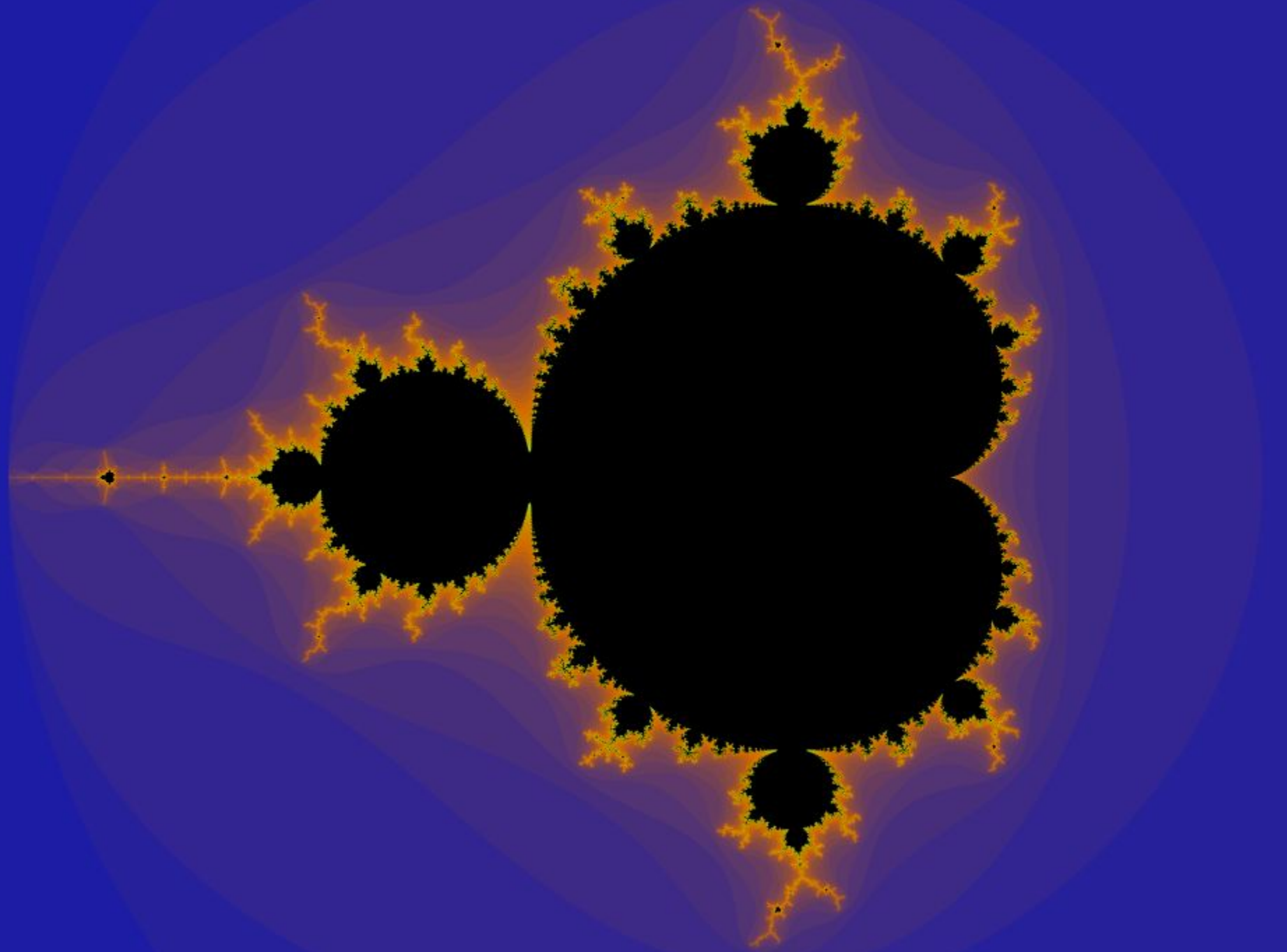

Customer Backend Service
1
Amazon API Gateway
AWS Step Functions (orchestrator)
2
Document Preprocessing
AWS Lambda (data validation)
Amazon SQS (pacing)
AWS Lambda (preprocessing)
Amazon S3 (processed docs bucket)
3
Intelligent Document Processing
AWS Lambda (classifier for digital vs. scanned)
AWS Lambda (fields extraction)
Amazon Textract (OCR)
Amazon Bedrock (ICR)
Amazon S3 (results bucket)
4
Doctors Verification
Amazon OpenSearch (registered doctors)
Amazon Bedrock (query & content parsing)
AWS Lambda (doctor search)
Amazon SQS (not found doctors)
5
Amazon SQS (parallel processing & concurrency control)
6
Format Verification
Amazon Bedrock
Amazon S3 (processed docs bucket)
AWS Lambda (known format verification)
AWS Lambda (comparison to previous formats)
7
Forgery Detection
Amazon SageMaker AI (compression & heatmap)
Amazon S3 (temp results storage)
AWS Lambda (metadata analysis)
AWS Lambda (sources of truth comparison)
8
Recurrence Rate Analysis
Amazon SageMaker AI
AWS Lambda (recurrence analysis)
Amazon DynamoDB (reimbursements table)
AWS Lambda (cost distribution analysis)
9
Amazon DynamoDB (results table)
10
Claims Insights Data Lake
AWS Glue (data sync job)
Amazon S3 (classified results)
AWS Athena
Amazon Quicksight (dashboards)
11
Classification & Feedback
Amazon SageMaker AI (classifier)
AWS Lambda (retro feedback)


PANALIGHT
SISTEMAS
DE VERDAD
imgflip.com


New Pipeline Architecture
Cloudflare Central Datacenter
Browser
Visitors
Data center 1
Data center 2
Data center 120+
Kafka Cluster
Broker 1
Logfwdr 1
Broker 2
Logfwdr 2
Broker 106
Logfwdr 106
6 M/s
6 M/s
Kafka consumers
(Marathon Go apps)
Consumer 1
Consumer 2
Consumer 106
a.k.a. Clickhouse inserters
ClickHouse cluster
Shard 1
Shard 2
Shard 36
40 q/s
Zone Analytics API
(Marathon Go apps)
Instance 1
Instance 2
Instance 5
{ api }
Load Balancer
API users


User
Postgres
DWH
User
DWH

Los problemas de la escala



Todos los hacks para solucionar los “edge cases” es lo que hace a tu sistema un buen sistema

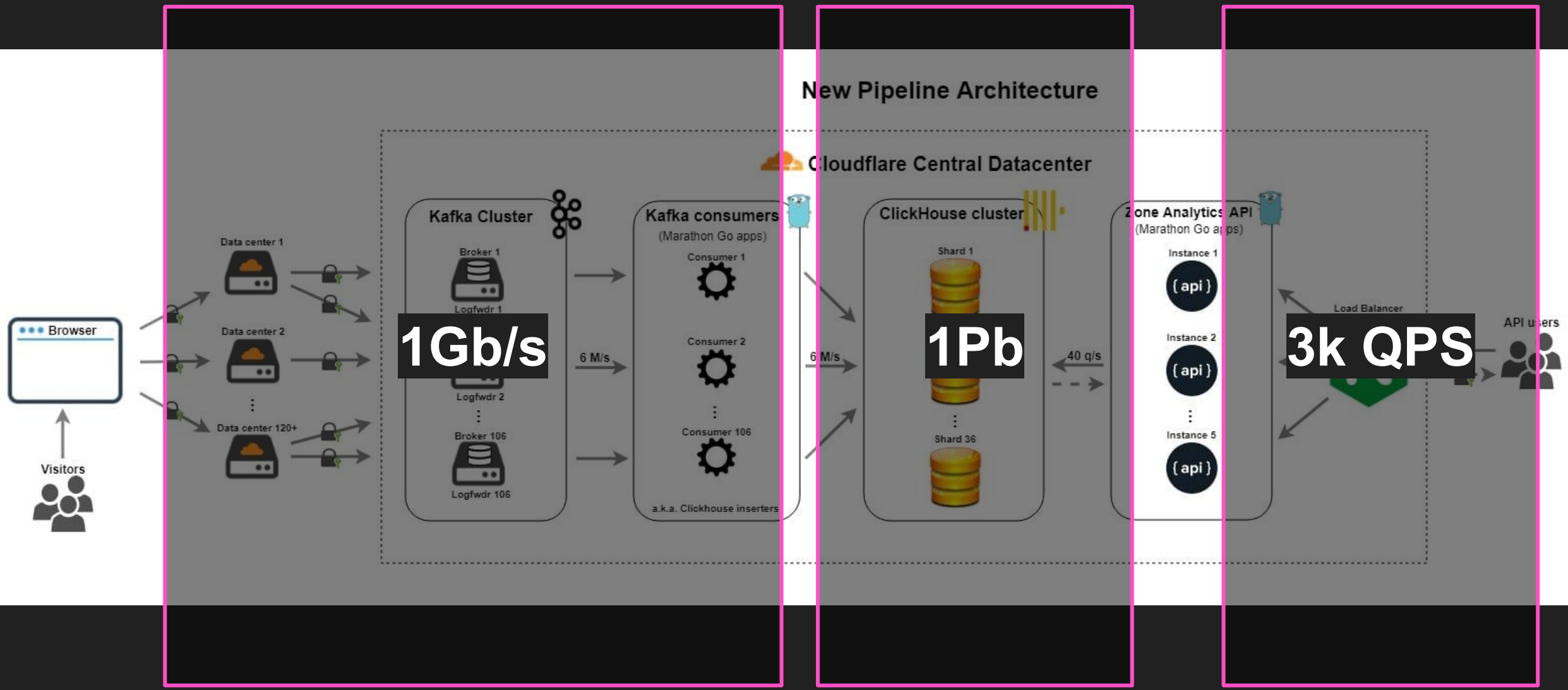
- *Santana, sacada de la manga*

Cualquier sistema que se acerque a su punto de carga máxima, lo hace de forma degradada.

- *Santana, adaptando la teoría de colas*

Qué es escala (más o menos)

+1 Pb
+1 Gb/s
3k QPS


New Pipeline Architecture
Cloudflare Central Datacenter
Browser
Visitors
Data center 1
Data center 2
Data center 120+
Kafka Cluster
Broker 1
Logfwdr 1
Logfwdr 2
Broker 106
Logfwdr 106
1Gb/s
6 M/s
Kafka consumers
(Marathon Go apps)
Consumer 1
Consumer 2
Consumer 106
a.k.a. Clickhouse inserters
6 M/s
ClickHouse cluster
Shard 1
Shard 36
1Pb
40 q/s
Zone Analytics API
(Marathon Go apps)
Instance 1
Instance 2
Instance 5
{ api }
Load Balancer
3k QPS
API users

Escalar no [solo] es ser capaz de poner más hardware o apilar servicios de AWS, vamos a ver el porqué

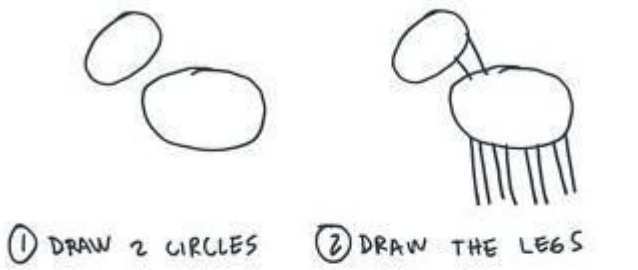


Los problemas de la escala

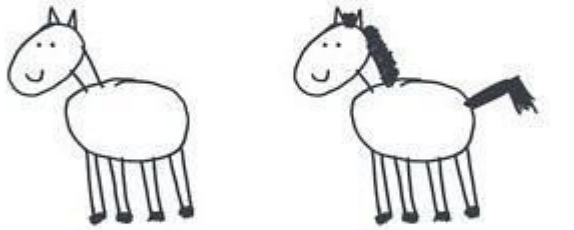
Ejemplos prácticos, del mundo real, [casi] sin cajitas

HOW TO:
DRAW A HORSE

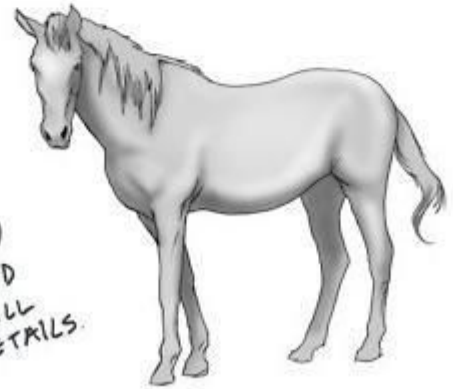
BY VAN OKTOP



① DRAW 2 CIRCLES ② DRAW THE LEGS



③ DRAW THE FACE ④ DRAW THE HAIR



⑤ ADD SMALL DETAILS.

No linearidades

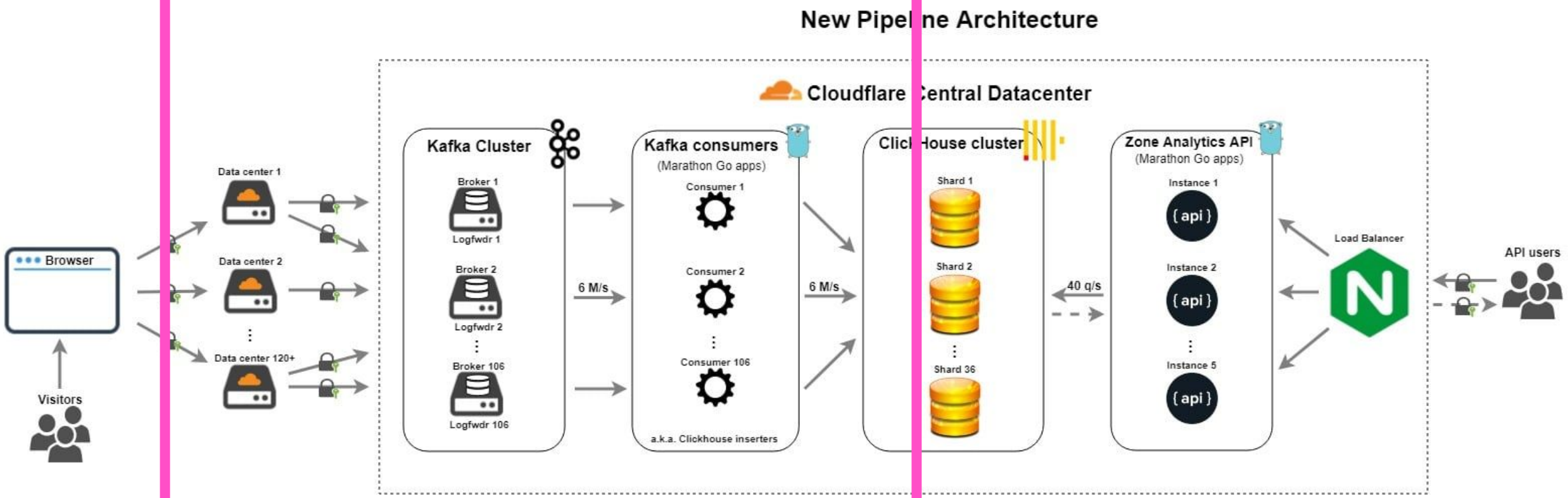
$O(n)$ vs $O(n^2)$

```
SELECT
    c1,
    count(distinct(c2))
FROM table
GROUP BY c1
```

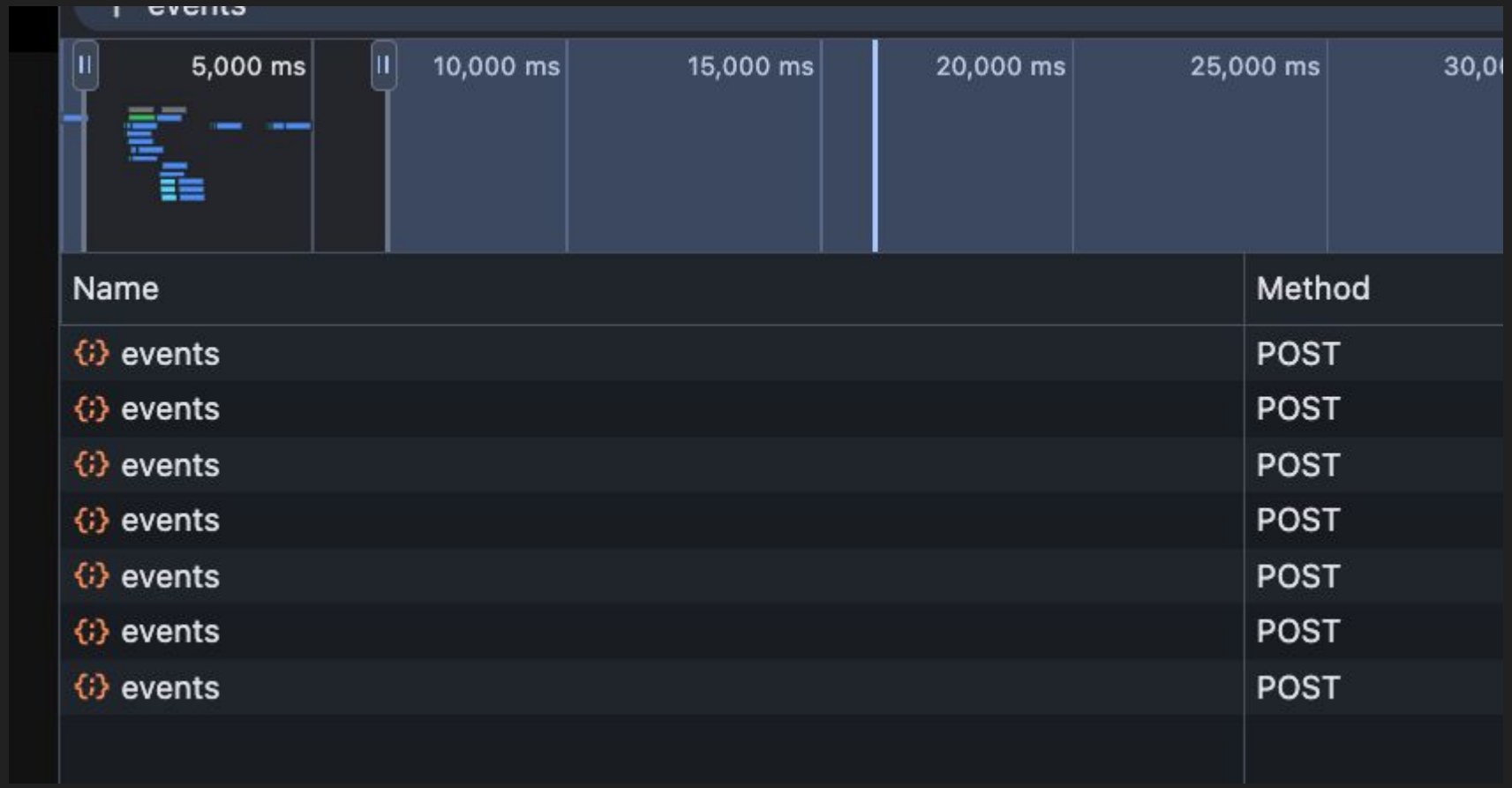
ser inteligentes >> añadir hardware

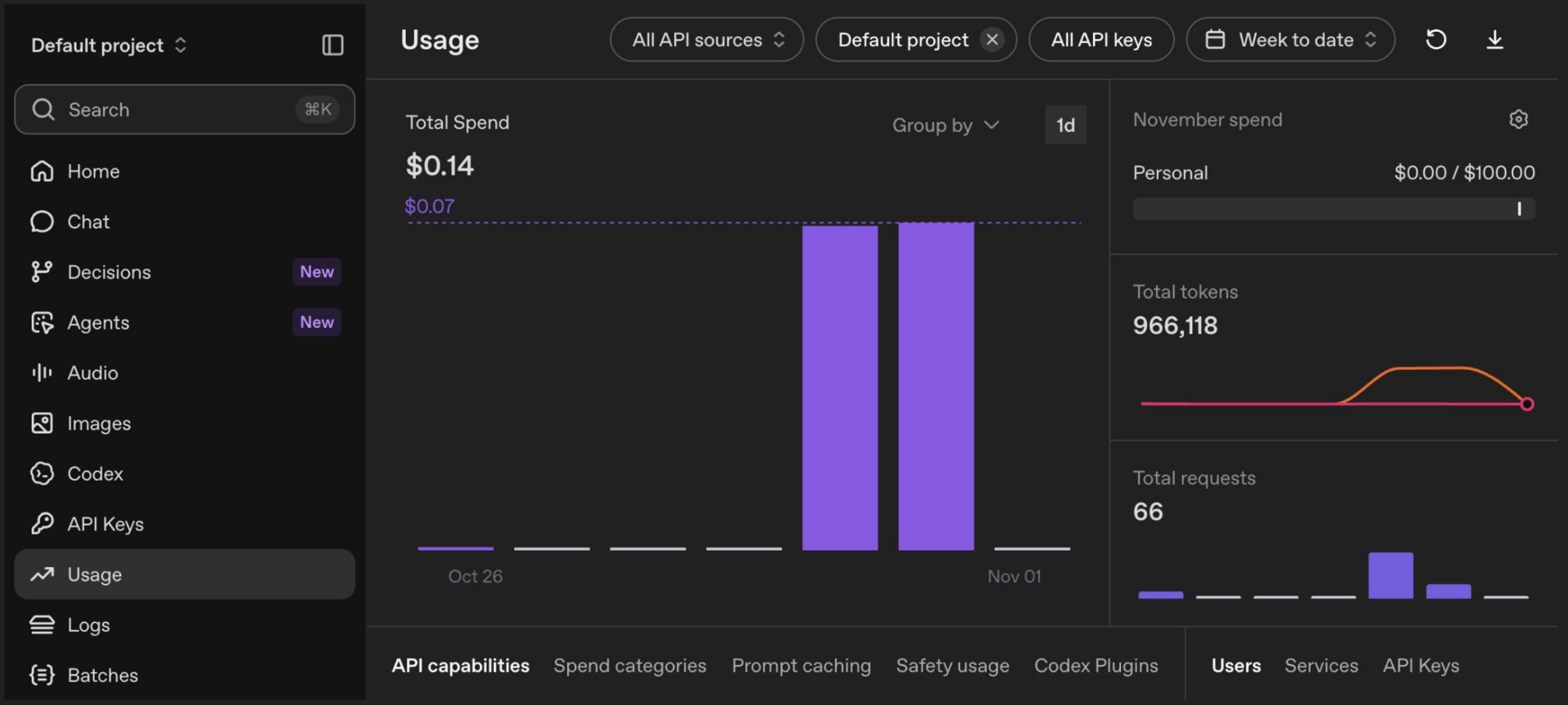
ser inteligentes >> añadir un nuevo servicio de GCP/AWS

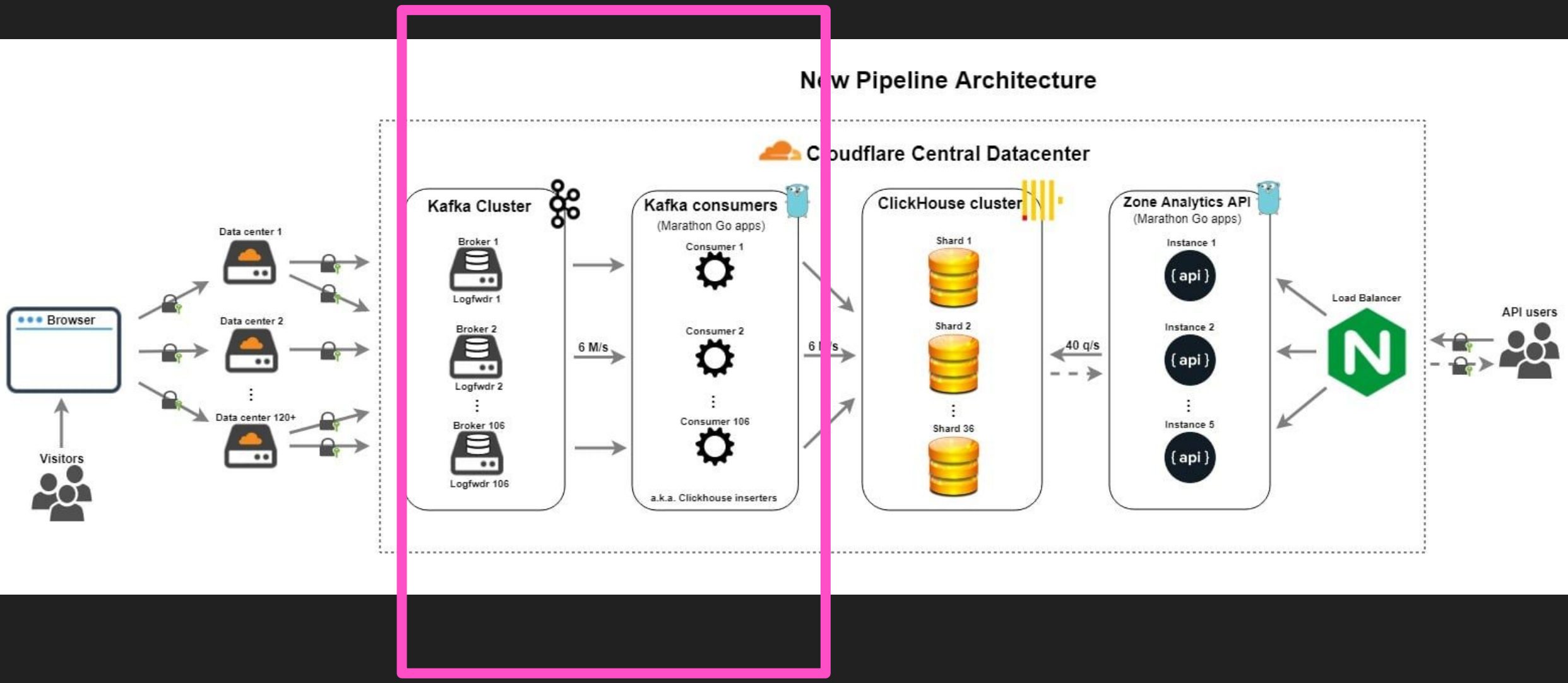
Ingesta


New Pipeline Architecture
Cloudflare Central Datacenter
Browser
Visitors
Data center 1
Data center 2
Data center 120+
Kafka Cluster
Broker 1
Logfwdr 1
Broker 2
Logfwdr 2
Broker 106
Logfwdr 106
6 M/s
6 M/s
Kafka consumers
(Marathon Go apps)
Consumer 1
Consumer 2
Consumer 106
a.k.a. Clickhouse inserters
ClickHouse cluster
Shard 1
Shard 2
Shard 36
40 q/s
Zone Analytics API
(Marathon Go apps)
Instance 1
Instance 2
Instance 5
{ api }
Load Balancer
API users

Batching vs latencia


events
5,000 ms
10,000 ms
15,000 ms
20,000 ms
25,000 ms
30,0
Name
Method
events
POST
events
POST
events
POST
events
POST
events
POST
events
POST
events
POST


Default project
Search ⌘K
Home
Chat
Decisions New
Agents New
Audio
Images
Codex
API Keys
Usage
Logs
Batches
Usage
All API sources
Default project
All API keys
Week to date
Total Spend
$0.14
$0.07
Group by
1d
Oct 26
Nov 01
November spend
Personal
$0.00 / $100.00
Total tokens
966,118
Total requests
66
API capabilities
Spend categories
Prompt caching
Safety usage
Codex Plugins
Users
Services
API Keys


New Pipeline Architecture
Cloudflare Central Datacenter
Browser
Visitors
Data center 1
Data center 2
Data center 120+
Kafka Cluster
Broker 1
Logfwdr 1
Broker 2
Logfwdr 2
Broker 106
Logfwdr 106
6 M/s
Kafka consumers
(Marathon Go apps)
Consumer 1
Consumer 2
Consumer 106
a.k.a. Clickhouse inserters
ClickHouse cluster
Shard 1
Shard 2
Shard 36
40 q/s
Zone Analytics API
(Marathon Go apps)
Instance 1
Instance 2
Instance 5
{ api }
Load Balancer
API users

Desacoplar la ingesta de la base de datos



Tobias Lins ▲
@tobiaslins

We've been using similar concepts when building **Vercel Data Pipeline**
- Processing over **5GB/s**
- Supports multi fanout
- Deduplication
- At-least once delivery

I think I need to write a blogpost

> 
> **Almog Gavra** @almoggavra · May 29
> I'm pretty sure the only reason to use Kafka for telemetry data in 2026 is muscle memory.
>
> A simple, open source, S3-backed pipeline can shuttle 1Gb/s of logs to @ClickHouseDB for $200/mo with 2.8s p50 latency. Fast enough for us....

7:57 PM · May 30, 2026 · **701.6K** Views

Cuanto preproceso?

Diseño de frontend

Google Analytics PPC Dashboard

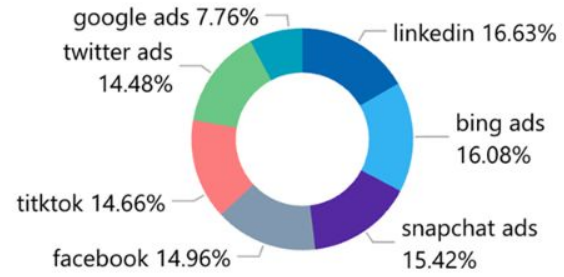
COUPLER.IO

Overview | Dynamics | Setup dashboard

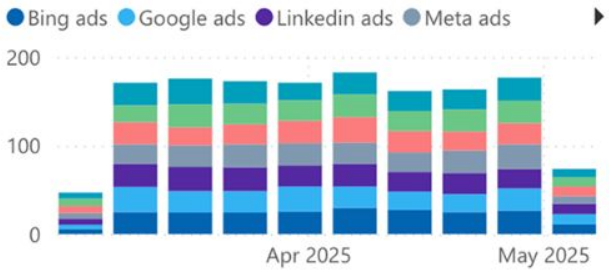
Total users	Sessions	Avg engmt time	Purchase rate	Purchases	Revenue
4.4K	5.4K	00:00:44	18.60%	813	2.8K

Impressions	Clicks	CTR	CPM	CPC	ROAS
453.8K	5.5K	1.22%	3.30	0.27	185.38%

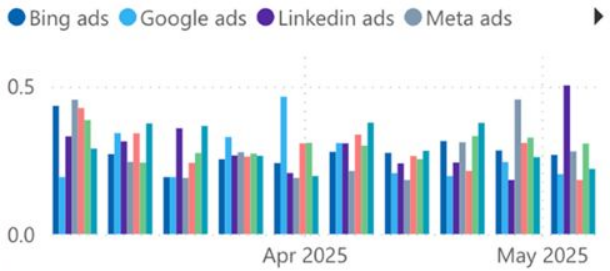
Traffic by channel



Weekly ad spend



Weekly CPC



Session source	Total users	Sessions	Engaged sessions	Purchases	Purchase rate
bing ads	703	870	429	128	18.21%
facebook	654	800	419	125	19.11%
google ads	339	431	234	66	19.47%
instagram	0	0	0	0	
linkedin	727	909	449	124	17.06%

Ad source	Ad spend	Impressions	Clicks	CPC	CPM	CTR
Bing ads	224	67.4K	851	0.26	3.32	1.26%
Google ads	213	66.0K	813	0.26	3.22	1.23%
Linkedin ads	216	59.3K	811	0.27	3.64	1.37%
Meta ads	212	69.1K	862	0.25	3.06	1.25%
Snapchat ads	211	63.6K	760	0.28	3.32	1.20%
TikTok ads	209	63.8K	716	0.29	3.27	1.12%

Report date

3/8/2025 — 5/7/2025

Ad source

All

Ad campaign

All

Session source

All

Session campaign

All

Have questions?



Dashboard setup guide

Book a demo

Contact support

Connect 70+ data sources with Coupler.io

DuckDB 2.0 agent mode

59% fewer tokens. The bill barely moved.

The real change is that missing rows are now visible.

Before

```
| customer_0    |     3 |    777 |
|       .       |   .   |    .   |
| customer_4999 |     2 |    510 |
| 5000 rows (40 shown)  3 columns |
```

Agent mode

```
| 19 | customer_19 | 13.3 |
| … 4960 rows omitted … |
| 4980 | customer_30 | 86.0 |
```

A model can skim past a footer. It can't skim past a row.

Source: duckdb.org/2026/10/09/agent-mode

Isin Pesch

Resiliencia: degradación y recuperación

A 10gb/s, 1 minuto de caída, te genera una montonera tremenda, así que tienes que poner sistemas guardarrailes

S3 se degrada, los discos se degradan, las máquinas se caen, los interfaces de red dejan de funcionar, las CPU tienen contención con determinadas cargas, la red se degrada, los vecinos ruidosos, GCP reinicia todas las máquinas al mismo tiempo sin previo aviso...

Qué degradas? inserciones, lecturas, procesos batch?

Cómo degradas? queries más lentas, 503, rate limiting, datos antiguos, priorizas ciertas queries

Evolución de esquemas

Los esquemas son **críticos**.

Esta es la pinta que tiene un esquema real

timestamp	
zoneId	
zonePlan	
ownerId	
zoneName	
hosterId	
brandId	
rayId	
securityLevel	
flags	
unstable	
unstablePublic	
rayId2	
tenantId	
connectionId	
ratePlan	
clientRequest	26 fields
edgeResponse	10 fields
edge	45 fields
edgeRequest	8 fields
cacheResponse	12 fields
cache	18 fields
cacheRequest	2 fields
originResponse	8 fields
origin	19 fields
backend	10 fields
edgeWorker	13 fields
argoSmRouteConfig	3 fields
argoTunnel	3 fields

Ojalá

AUGUST 27, 2026

How we saved 100 terabytes of memory by optimizing 1.1.1.1's DNS cache



Sebastiaan Neuteboom

12 MINUTE READ COPY URL

https://blog.cloudflare.com/dns-cache-memory-optimization-1111/

```
CREATE TABLE events (

    …
    device_type Int32
    …
)
```

```
CREATE TABLE events (

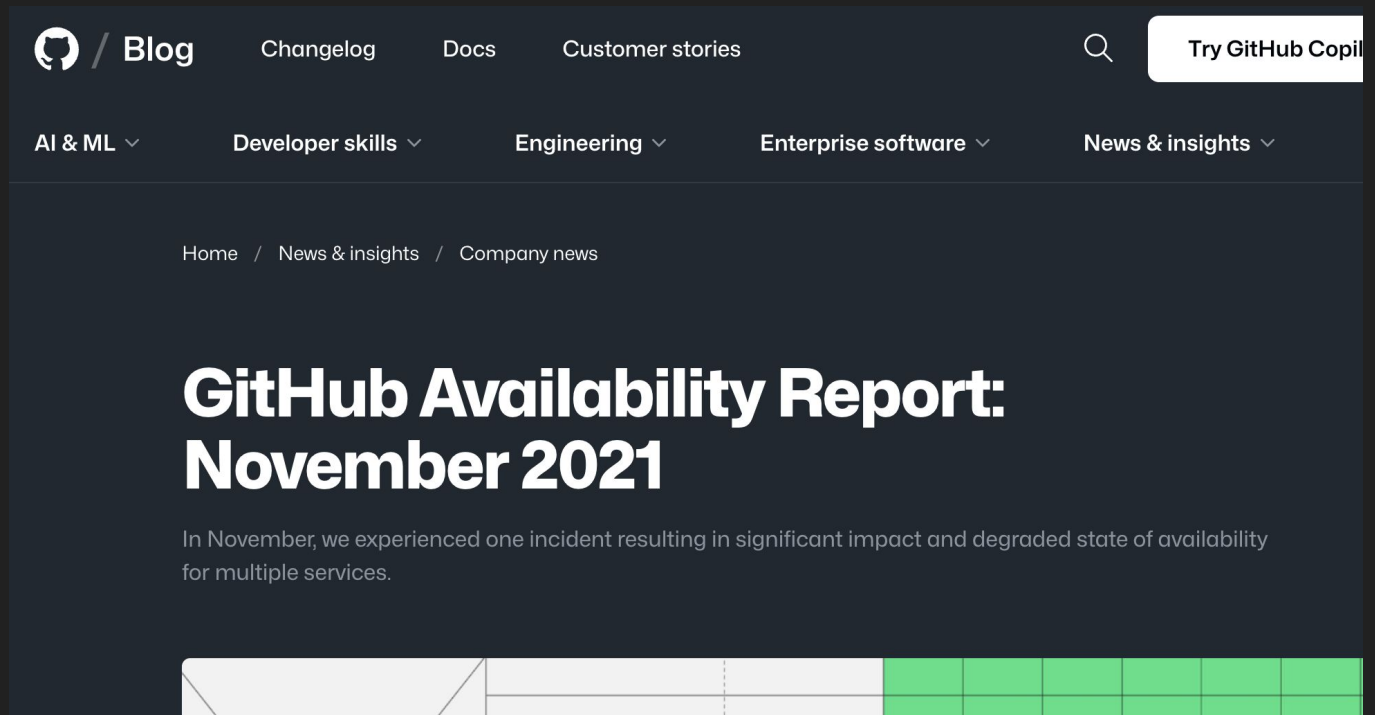
    …
    device_type Int8
    …
)
```

1M rows -> 4Mb vs 1Mb
1T rows -> pues eso

Obviamente, las cosas no son tan obvias: sorting, compression, data types, encoding, TTL, shredding

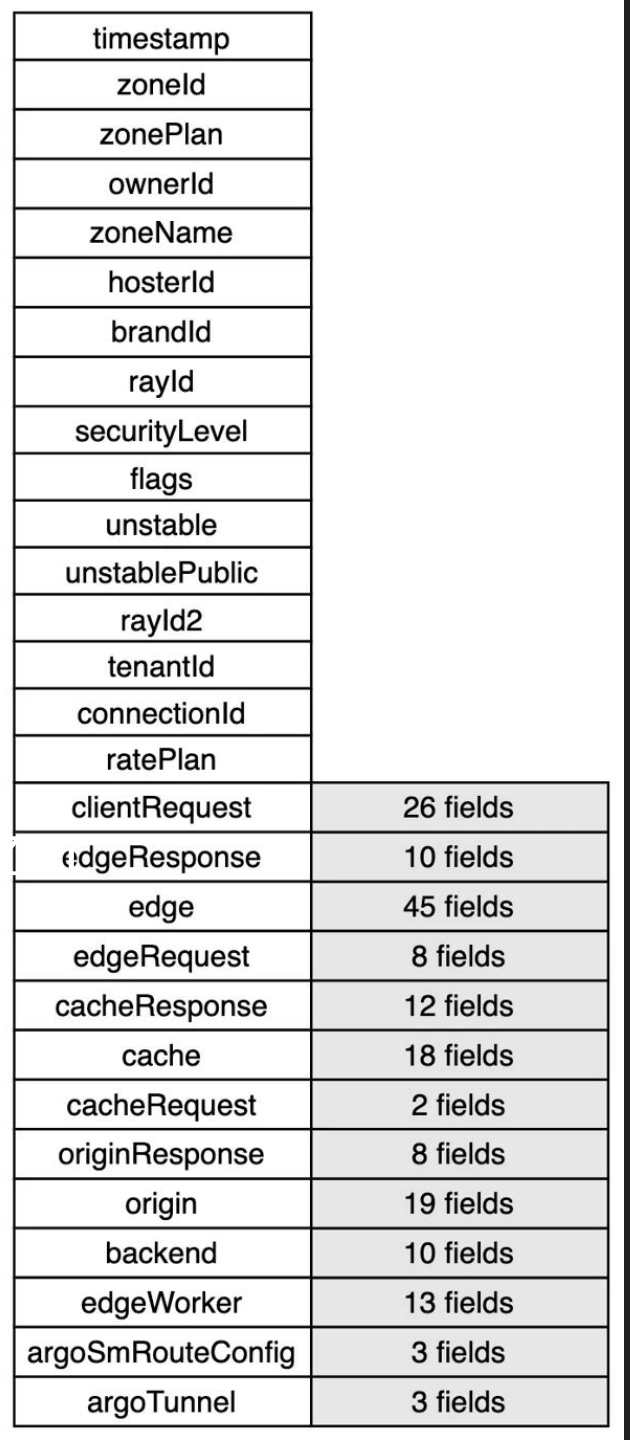
Y no hablemos de migraciones. Normalmente es “añadir una columna” pero a veces es todo el stack de ingesta completo.

Un cambio de esquema puede tirar el sistema abajo



Denormalizar²

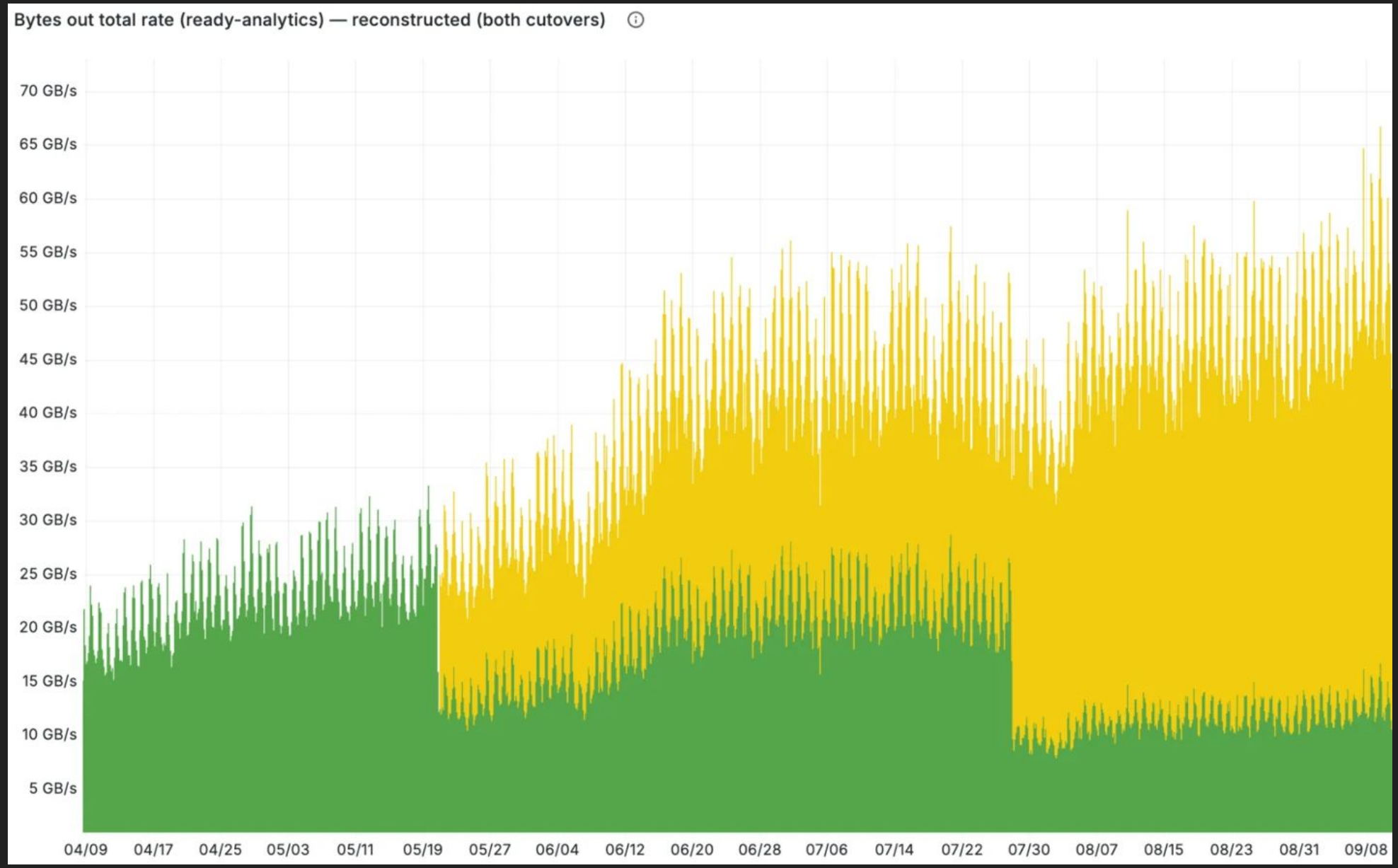
star schema -> denormalizar -> denormaliz



timestamp
zoneId
zonePlan
ownerId
zoneName
hosterId
brandId
rayId
securityLevel
flags
unstable
unstablePublic
rayId2
tenantId
connectionId
ratePlan

clientRequest	26 fields
edgeResponse	10 fields
edge	45 fields
edgeRequest	8 fields
cacheResponse	12 fields
cache	18 fields
cacheRequest	2 fields
originResponse	8 fields
origin	19 fields
backend	10 fields
edgeWorker	13 fields
argoSmRouteConfig	3 fields
argoTunnel	3 fields

Gestionar crecimiento exponencial



https://blog.cloudflare.com/rewinding-the-exponential/

Observabilidad

La propia observabilidad del es muy compleja y muy cara

La base de datos “no es una pieza más”

03 Technical deep dive

```
void BloomFilter::add(const char * data, size_t len)
{
    size_t hash1 = CityHash_v1_0_2::CityHash64WithSeed(data, len, seed);
    size_t hash2 = CityHash_v1_0_2::CityHash64WithSeed(data, len, SEED_GEN_A * seed + SEED_GEN_B);

    for (size_t i = 0; i < hashes; ++i)
    {
        size_t pos = fastMod(hash1 + i * hash2 + i * i);
        filter[pos / (8 * sizeof(UnderType))] |= (1ULL << (pos % (8 * sizeof(UnderType))));
    }
}
```



https://www.youtube.com/watch?v=yIVzoNKwQvA

1 trillion rows was a challenge

DuckLake at PostHog didn't start small and ramp up. We fully committed and put all of PostHog in a DuckLake to dogfood as the first customer.

When we moved off it, it held about 2 PB: roughly 1 trillion rows and 15.3 million snapshots. It broke in lots of ways that were hard to fix within DuckLake's scope.

We'd rather have contributed to the existing community and helped it grow. But we couldn't land changes as fast as we needed, so we ended up carrying patches in a fork which defeats one of the main benefits of using DuckLake: the community.

https://posthog.com/blog/moving-off-duckdb

Outliers

Hay un 5% de tus clientes que se llevan un 95% del uso.

Tienes que diseñar el sistema para ellos, no vale tratarlos como un outlier (o sí)

Eventos masivos y picos

shopify

ENGINEERING BLOG
Menu

BLOG | INFRASTRUCTURE

How we prepare Shopify for BFCM

From March to October we simulated traffic tsunamis, injected chaos, and fixed every bottleneck before our merchants needed us most.

Building year-round resilience

Our BFCM prep started in March with capacity planning and a multi-region strategy on Google Cloud. Reliability is year-round work. We don't use BFCM as a release deadline—every architectural change and migration happens months before that critical window.

DDoS, ataques, eventos inesperados, benchmarks, deploys...

Costes

S3 storage and ops
Balance latencia vs coste (más memoria, cachés)
Menos máquinas grandes vs muchas pequeñas
El atraco de AWS y GCP - egress
Servicio managed vs construir
Spot instances
Local SSD
El atraco de AWS y GCP - egress
CPU architecture (ARM, Intel y variantes)
Autoscaling (o no y con que margen)
Asignación de costes
El atraco de AWS y GCP - egress

Control de recursos
Seguridad
Gestión de concurrencia
Observabilidad
Lineage
Auditorías
Backups

Despedida y cierre

Honestamente no sé qué pinta tendrán la infra [de datos] “IA native” pero…

Median end-to-end milliseconds, 21 runs, warm cache, one thread:

Query	analytics.c	DuckDB	ClickHouse local
Summary	**29.6**	92.9	189.9
Landing before/after	**26.3**	63.9	184.1
Top pages	**30.2**	94.1	186.9
Landing clicks	**26.2**	58.7	180.2

ClickHouse is much closer without process startup: its warm queries take **29–39 ms**, versus **23–27 ms** inside C—about a **1.2–1.4× advantage for C.**

Manuel Cánovas García
José Luis Vilar Hueso
Salvador Aznar Calabuig
Claudio Gómez-Perretta
Eustaquio Berriochoa
Félix Amorena



Gracias

@javisantana